

基于德英翻译任务的 Transformer 复现与位置编码对比研究

杨帆

23375012@buaa.edu.cn

摘要

本文以德语到英语的机器翻译任务为研究对象，从零复现 Transformer Encoder-Decoder 结构，并在此基础上开展位置编码单因素对比实验，分析不同位置编码方式对模型序列建模性能的影响。实验数据为 Tatoeba 项目提供的德英平行句对，去重后包含超过 30 万对平行句，文本长度跨度较大，具有明显序列依赖和复杂语法结构。正式实验从去重语料中抽取 100,000 对平行句，并按照 80%、10%、10% 划分为训练集、验证集和测试集；模型输入统一采用词嵌入，并根据实验组设置是否加入相应位置编码，序列长度限制为 30 个 token。

本文基于 PyTorch 实现了 Transformer Encoder-Decoder 结构，并设计 none、absolute、sinusoidal、learned 和 gated_mix 五组位置编码实验。除位置编码外，五组实验的网络结构、训练参数和优化策略保持一致，以确保比较公平。模型训练 20 epoch，采用 Adam 优化器，损失函数为忽略填充位置的交叉熵，验证集损失用于选择最优模型，测试集 token accuracy 和 BLEU 作为主要评价指标，同时按短句、中句和长句分桶评价，并结合翻译样例进行定性分析。

实验结果表明，五组方法均能捕捉序列基本结构，其中 learned 位置编码在整体 token accuracy、BLEU 及各句长分桶上均取得最优表现；sinusoidal 在长句分桶上优于 absolute，体现多频正弦编码在长距离依赖建模中的优势；absolute 性能较弱，说明线性固定位置表示对复杂句子建模存在局限；gated_mix 虽体现一定自适应能力，但整体未超过 learned。分析显示，模型在短句和常见结构句子中表现相对稳定，而长句和复杂结构仍受位置编码影响明显。

综上，本文验证了位置编码在 Transformer 序列建模中的核心作用。learned 整体表现最优，sinusoidal 在长句分桶上体现相对优势，absolute 存在一定的局限性，gated_mix 体现了轻量融合潜力。

关键词：Transformer；机器翻译；位置编码；自注意力机制；德英翻译；PyTorch

一、引言

机器翻译是自然语言处理中的经典任务，其目标是将一种自然语言中的句子自动转换为另一种自然语言中的等价表达。传统的序列到序列模型通常基于循环神经网络（RNN）或长短期记忆网络（LSTM）^[3]构建，通过沿时间步递推的方式处理输入和输出序列。这类方法能够在一定程度上刻画上下文信息，但其循环结构限制了训练过程的并行计算能力，并且在处理长序列时容易面临梯度传播困难和长距离依赖捕捉不足等问题。

2017 年，Vaswani 等人在论文《Attention Is All You Need》中提出 Transformer 模型^[1]。在此之前，注意力机制已经被用于神经机器翻译任务中，使解码器能够在生成目标语言时动态关注源语言序列的不同位置，从而缓解固定长度上下文表示带来的信息瓶颈^[2]。与 RNN 和 LSTM 不同，Transformer 进一步将注意力机制作为模型的核心结构，完全摒弃循环结构，主要依靠自注意力机制对序列中不同位置之间的关系进行建模。自注意力机制能够直接计算序列中任意两个 token 之间的相关程度，使每个位置都可以在一次计算中获取全局上下文信息。因此，Transformer 不仅提高了训练并行性，也增强了模型对长距离依赖关系的表达能力，在机器翻译等序列建模任务中表现出较强优势。

然而，自注意力机制本身并不包含显式的词序信息。在不引入位置编码的情况下，注意力计算主要根据 token 表示之间的相似度分配权重，而无法天然地区分 token 在原始句子中的先后位置。对于自然语言而言，词序往往直接影响句法结构和语义含义。例如，同一组词在不同排列顺序下可能表达完全不同的意思。因此，若 Transformer 只依赖词嵌入和注意力权重，而不额外加入位置信息，模型对序列结构的理解就会受到限制。

为解决这一问题，原论文在词嵌入向量中加入位置编码，使模型在处理每个 token 时同时获得其语义信息和位置信息。位置编码的设计方式并非唯一，不同编码方式可能对模型性能产生不同影响。原论文采用固定的正弦位置编码，它不引入额外可训练参数，并能够通过不同频率的正弦和余弦函数表示不同位置；而在实际应用中，也可以采用可学习位置编码，直接让模型从数据中学习每个位置的向量表示。此外，若仅使用简单的绝对位置编码，也可以检验粗略位置信息是否足以支撑 Transformer 完成翻译任务。

本文是模式识别与机器学习课程第四次作业的实验报告。根据作业要求，本文首先复现经典 Transformer Encoder-Decoder 结构，并在此基础上选择位置编码方向开展模块必要性实验。本文的研究重点包括：不同位置编码方式在德英翻译任务中的表现差异；简单绝对位置编码与原论文正弦位置编码之间的效果差别；可学习位置编码是否能够在当前数据规模下获得更适合任务的位置表示；以及门控混合位置编码能否通过融合正弦编码和简单绝对编码获得额外收益。通过上述实验，本文希望从训练过程、测试集指标、句长分桶结果和翻译样例等多个角度分析位置编码在 Transformer 机器翻译模型中的作用。

二、数据集说明与预处理

2.1 数据来源与格式

本文使用的数据集为 Tatoeba 项目提供的德英平行句对话料，原始文件为 data/deu.txt^[5]。该文件中每一行对应一个平行句对，主要包含英语句子、德语句子 and 版权信息三列，并以制表符进行分隔。本文任务方向设定为德语到英语翻译，即德语句子作为源语言输入，英语句子作为目标语言输出。

原始数据共包含 324,282 行平行句对。由于部分句对可能存在重复，为避免重复样本对训练和评价结果造成影响，本文在预处理阶段进行了大小写不敏感的去重处理，最终保留 324,230 对平行句。整体来看，该数据集格式较为规整，无明显空行和异常格式行，适合作为 Transformer 机器翻译复现实验的数据基础。

2.2 数据分布与清洗

在正式建模之前，本文首先对原始句子进行基础清洗和分词处理。具体而言，将德语句子和英语句子统一转换为小写，并采用空格分词方式将句子切分为 token 序列。虽然空格分词无法像 BPE 等子词方法那样细致处理德语复合词和低频词，但其实现简单、可解释性较强，适合本文从零复现 Transformer 结构和开展位置编码对比实验的目标。

在词表构建方面，本文分别为源语言德语和目标语言英语建立词表，并加入四个特殊符号：<pad>、<bos>、<eos>和<unk>。其中，<pad>用于对不同长度句子进行填充，<bos>用于标记目标端解码开始，<eos>用于标记句子结束，<unk>用于表示未登录词。训练时设置最大序列长度 max_len=30，对于超过该长度的

句子进行截断，对于较短句子则使用<pad>补齐，以便批量输入模型。

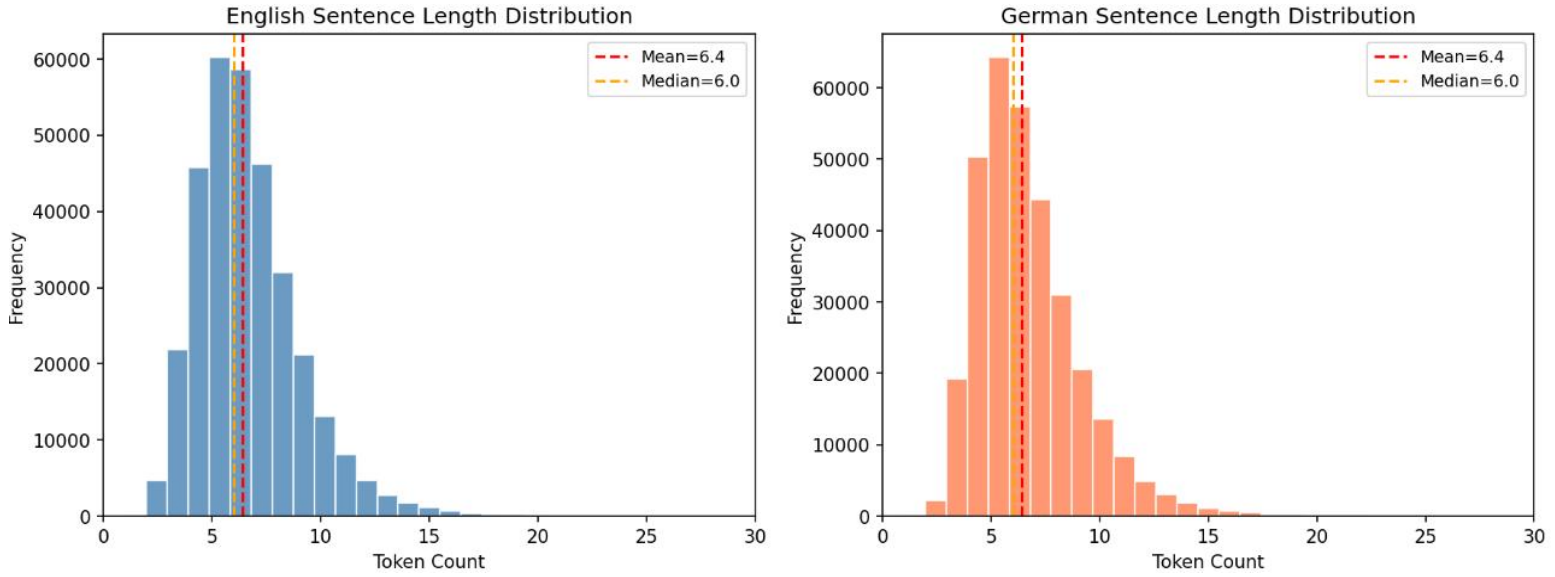


图 1 德英句子长度分布

图 1 展示了去重后德英句子的长度分布。由图可见，德语和英语句子的长度分布较为接近，整体呈现明显的长尾特征。绝大多数句子集中在 1 到 10 个词范围内，其中 6 到 10 个词区间的样本数量最多，10 个词以下的句子合计占比超过 90%。两种语言句长均值约为 6.4 个词，中位数均为 6 个词。这说明该语料以短句和中等长度句子为主，适合在有限算力条件下进行 Transformer 复现实验。

虽然长句在数据集中所占比例较低，但它们仍然具有重要分析价值。原因在于，长句往往包含更复杂的语法结构和更长距离的词间依赖，对模型的序列建模能力要求更高。相比短句，长句更容易暴露位置编码设计的差异。因此，本文后续不仅统计整体 token accuracy，还进一步按照参考译文长度划分短句、中句和长句三个分桶，以观察不同位置编码方式在不同句长条件下的性能变化。

2.3 实验数据划分

本文正式实验从去重后的数据集中随机抽取 100,000 对平行句作为实验样本。随后按照 80%、10%、10% 的比例划分为训练集、验证集和测试集，其中训练集包含 79,994 对句子，验证集包含 9,999 对句子，测试集包含 10,000 对句子。划分过程中固定随机种子 seed=42，以保证五组位置编码实验使用完全一致的数据划分，从而提高实验结果的可比性。

在正式实验中，最终源语言德语词表大小为 30,000，目标语言英语词表大小为 22,259。训练集用于模型参数学习，验证集用于监控训练过程并选择验证损失

最低的 checkpoint，测试集仅用于最终性能评价。需要特别说明的是，本文报告中的 valid_loss 来自验证集，而最终的 token_acc、short_acc、medium_acc 和 long_acc 均是在测试集上计算得到。这样处理可以避免直接根据测试集选择模型参数，使实验评价过程更加规范。

此外，本文在正式实验之前还进行了两轮前期验证训练。第一轮使用 8K 样本、5epoch 和 d_model=128 的轻量配置，主要用于验证数据管线、模型结构和训练流程是否能够正常运行。第二轮使用 50K 样本、12epoch 和 d_model=256 的中等规模配置，主要用于评估 GPU 训练速度、显存占用和结果提升趋势。上述两轮实验为后续正式实验提供了调试和参数选择依据，但本文正文分析以 100K 样本、20epoch 的正式实验结果为主。

三、Transformer 模型方法

本文从零实现了 Transformer Encoder-Decoder 结构，用于完成德语到英语的机器翻译任务。与基于循环结构的序列模型不同，Transformer 不再依赖 RNN 或 LSTM 逐步递推处理序列，而是通过自注意力机制直接建模序列中任意两个位置之间的关系。本文实现的模型主要包括词嵌入、位置编码、多头注意力、位置全连接前馈网络、残差连接、层归一化、Encoder、Decoder、Mask 机制以及训练和推理流程。以下对模型的关键组成部分进行说明。

3.1 序列到序列建模与整体框架

机器翻译属于典型的序列到序列任务。设源语言德语句子表示为

$$X = (x_1, x_2, \dots, x_m)$$

目标语言英语句子表示为

$$Y = (y_1, y_2, \dots, y_n)$$

模型的目标是学习从源语言序列到目标语言序列的条件概率分布，即在给定源句 X 的条件下生成目标句 Y。这一过程可以写为求解如下的条件概率值：

$$P(Y|X) = \prod_{t=1}^n P(y_t | y_1, y_2, \dots, y_{t-1}, X)$$

该公式表明，目标句并不是一次性整体生成的，而是按照自回归方式逐词生成。生成第 t 个目标词时，模型不仅需要利用源语言输入 X，还需要利用此前已

经生成的目标端前缀 $y_1, y_2, \dots, y_{n-1}$ 。Transformer 采用 Encoder-Decoder 框架完成上述映射。Encoder 负责对源语言序列进行编码，将德语输入转换为一组上下文表示；Decoder 则在 Encoder 输出的基础上，根据已经生成的目标端前缀逐步预测下一个英语 token。整体流程可以概括为：源语言德语句子经过词嵌入和位置编码后输入 Encoder，Encoder 通过多层自注意力和前馈网络提取上下文表示；Decoder 接收目标端已生成前缀，先通过 Masked Self-Attention 建模目标端历史信息，再通过 Encoder-Decoder Cross-Attention 读取源语言上下文，最后经线性层和 softmax 得到目标词表上的概率分布。

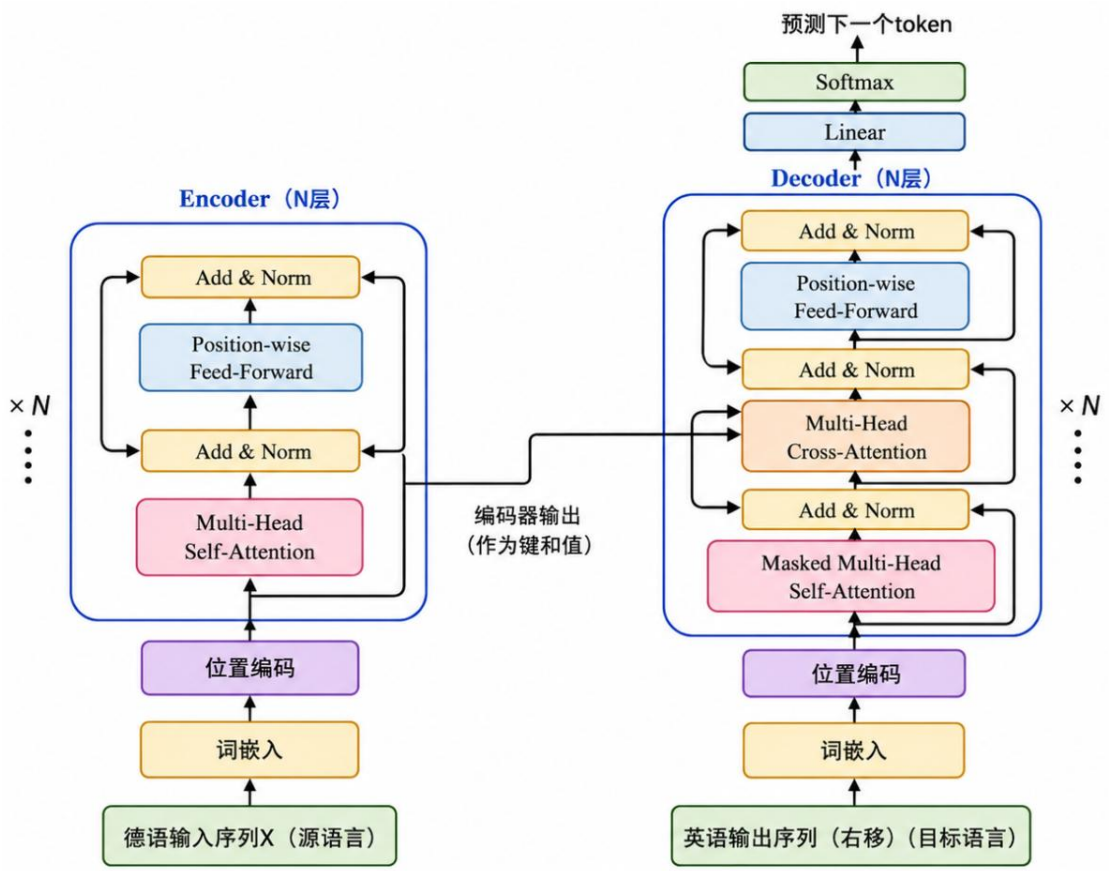


图 2 Transformer 整体架构示意图

图2给出了本文所复现的Transformer整体结构。该模型由Encoder和Decoder两部分组成，其中Encoder负责对源语言句子进行上下文编码，将德语输入序列转换为包含全局语义信息的隐藏表示；Decoder则在目标端已生成前缀和Encoder输出的共同作用下，逐步预测目标语言 token。与RNN或LSTM沿时间步递推建模不同，Transformer通过注意力机制直接建立序列中不同位置之间的联系，因此能够更有效地进行并行训练，并在长距离依赖建模方面具有更强的表达能力。

3.2 词嵌入、位置编码与输入表示

在进入 Encoder 和 Decoder 之前，离散的 token 索引首先需要经过词嵌入层映射为连续向量。对于源语言输入序列 X ，词嵌入层将每个 token x_i 转换为 d_{model} 维向量。词嵌入的作用在于将离散符号转化为连续表示，使模型能够在向量空间中学习词语之间的语义关系。

然而，单纯词嵌入只表示 token 本身的语义信息，并不包含 token 在句子中的位置。由于 Transformer 的自注意力机制本身不具备天然的顺序感知能力，因此需要在词嵌入上加入位置编码。本文采用的输入表示可写为

$$H_0 = \sqrt{d_{model}} E(X) + P$$

其中， $E(X)$ 表示词嵌入矩阵， P 表示位置编码矩阵， d_{model} 为模型隐藏维度。乘以 d_{model} 的目的是调整词嵌入的数值尺度，使其与位置编码相加后保持较稳定的表示范围。经过该步骤后，模型输入同时包含 token 语义信息和词序位置信息。

对于 Decoder 端，目标语言输入同样需要经过词嵌入和位置编码。训练阶段 Decoder 输入为目标序列右移一位后的结果，即在句首加入 $\langle \text{bos} \rangle$ ，并以真实目标词作为前缀输入；推理阶段则从 $\langle \text{bos} \rangle$ 开始，由模型逐步生成后续 token。

3.3 Encoder 结构

Encoder 由若干个结构相同的 Encoder Layer 堆叠而成。本文正式实验中采用 3 层 Encoder。每个 Encoder Layer 主要包含两个子层：多头自注意力子层和位置全连接前馈网络子层。每个子层外部都配有残差连接和层归一化，以保证深层模型训练过程稳定。

对于第 1 层 Encoder，若输入表示为 $H^{(l-1)}$ ，则其计算过程可以概括为

$$\tilde{H}^{(l)} = \text{LayerNorm}(H^{(l-1)} + \text{MultiHeadSelfAttention}(H^{(l-1)}))$$

$$H^{(l)} = \text{LayerNorm}(\tilde{H}^{(l)} + \text{FFN}(\tilde{H}^{(l)}))$$

在 Encoder 的自注意力计算中，序列中每个位置都可以关注源语言句子中的所有其他位置。因此，某个词的最终表示并不仅由该词本身决定，而是融合了整句上下文信息。例如，在德语句子里，一个名词的翻译可能依赖前后的冠词、动词和修饰成分；通过自注意力机制，Encoder 能够在 1 层内直接建立这些词之间的联系，而不需要像 RNN 那样沿时间步逐个传递信息。

Encoder 最终输出仍然与源语言序列等长。若源语言长度为 m ，则 Encoder 输出可以表示为

$$C = (c_1, c_2, \dots, c_m)$$

其中， c_i 表示源语言第 i 个位置的上下文表示。该输出将作为 Decoder 中 Cross-Attention 的键和值，用于后续目标语言生成。

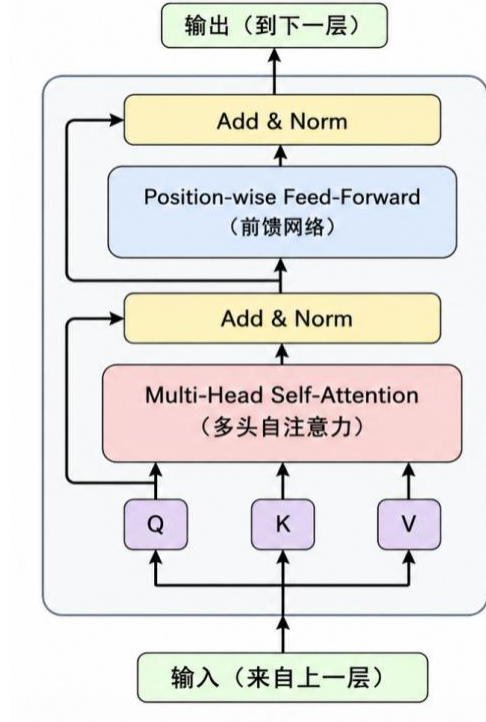


图 3 Encoder Layer 内部结构

从图 3 所示的 Encoder Layer 结构可以看出，单层 Encoder 主要由多头自注意力子层和位置全连接前馈网络子层组成，每个子层外部均配有残差连接和层归一化。多头自注意力子层用于在源语言序列内部建立不同 token 之间的依赖关系，使每个位置的表示能够融合全句上下文信息；前馈网络子层则对每个位置的表示进行进一步非线性变换，增强模型的特征表达能力。残差连接和层归一化的加入有助于稳定训练过程，使多层 Encoder 能够有效堆叠。

3.4 Decoder 结构

Decoder 同样由若干个结构相同的 Decoder Layer 堆叠而成。与 Encoder 不同，Decoder 每层包含三个子层：Masked Multi-Head Self-Attention、Encoder-Decoder Cross-Attention 和 Position-wise Feed-Forward Network。每个子层之后同样使用残差连接和层归一化。

Decoder 的第一子层是带掩码的自注意力层。该层只允许当前位置关注当前位置及之前已经出现的目标端 token，而不能关注未来 token。这样处理是为了保证训练阶段和推理阶段的一致性，即模型在预测第 t 个词时只能使用 $y_1, y_2, \dots, y_{t-1}$ ，不能提前看到真实答案中的未来部分。

Decoder 的第二子层是 Encoder-Decoder Cross-Attention。该层是 Transformer 完成机器翻译任务的关键。在 Cross-Attention 中，查询矩阵 Q 来自 Decoder 当前隐藏状态，键矩阵 K 和值矩阵 V 来自 Encoder 输出。也就是说，Decoder 在生成目标语言 token 时，会根据当前目标端上下文主动从源语言表示中检索相关信息。其基本形式可写为

$$\text{CrossAttention}(Q_{dec}, K_{enc}, V_{enc}) = \text{softmax}\left(\frac{Q_{dec}K_{enc}^T}{\sqrt{d_k}}\right)V_{enc}$$

该机制在功能上类似于翻译过程中的“对齐”：当 Decoder 生成某个英语词时，它可以动态关注德语源句中在当前生成词最相关的词或短语。与传统 Seq2Seq 模型中使用单一上下文向量不同，Transformer Decoder 可以在每个生成位置重新计算对源语言各位置的关注权重，因此具有更强的灵活性。

Decoder 的第三子层为前馈网络，用于对每个目标端位置的隐藏表示进行非线性变换。经过多层 Decoder 堆叠后，最终输出会通过线性层映射到目标语言词表大小，再经过 softmax 得到每个候选 token 的概率。

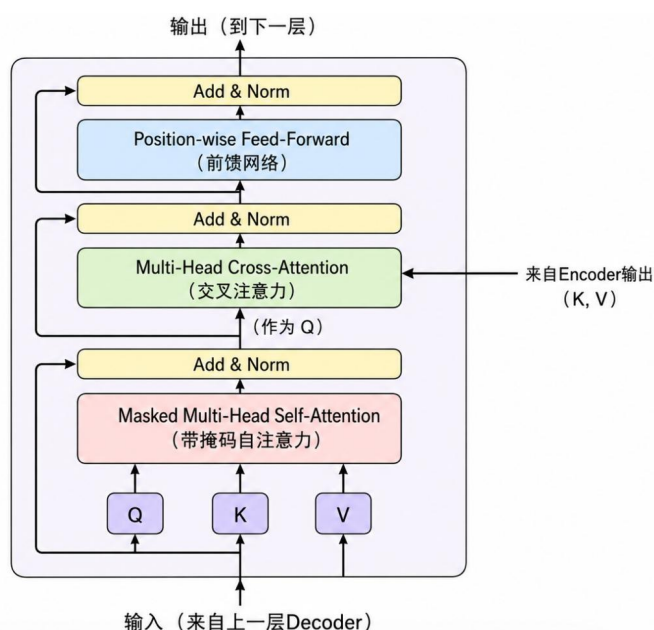


图 4 Decoder Layer 内部结构

图 4 展示了 Decoder Layer 的内部计算流程。相比 Encoder Layer，Decoder Layer 增加了 Encoder-Decoder Cross-Attention 子层。Decoder 首先通过 Masked Self-Attention 建模目标端已生成 token 之间的关系，并利用遮罩机制避免当前位置看到未来 token；随后通过 Cross-Attention 读取 Encoder 输出的源语言上下文表示，使目标端生成过程能够动态关注源句中相关位置。该结构使 Decoder 不仅依赖目标端历史信息，还能够结合源语言语义完成翻译生成，是 Transformer 实现序列到序列转换的关键环节。

3.5 Multi-Head Attention 机制

自注意力机制是 Transformer 的核心。给定输入序列的表示矩阵，模型首先通过三组可学习线性变换得到查询矩阵 Q 、键矩阵 K 和值矩阵 V 。其中，查询向量表示当前位置希望寻找什么信息，键向量表示各位置提供什么索引信息，值向量则表示真正被加权汇总的内容。Scaled Dot-Product Attention 的计算公式为^[1]

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

其中， d_k 表示键向量维度。若不进行缩放，当维度较大时， QK^T 的数值可能过大，使 softmax 函数输出过于集中，从而导致梯度变小。除以 $\sqrt{d_k}$ 能够缓解这一问题，使训练过程更加稳定。

单头注意力只能在一个表示子空间中计算词间关系，而自然语言句子中的依赖关系往往具有多样性。为此，Transformer 采用多头注意力机制，将 Q, K, V 分别投影到多个子空间中，并行计算多个注意力头。第 i 个注意力头可表示为

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

多个注意力头的输出经过拼接后，再通过线性映射得到最终输出：

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

多头注意力的优势在于，不同注意力头可以从不同角度捕捉序列关系。有的头可能更关注相邻词之间的局部句法关系，有的头可能更关注长距离依赖，有的头则可能学习到动词与宾语、主语与谓语等语义联系。多个头的信息经过拼接和融合后，模型能够获得比单头注意力更加丰富的上下文表示。

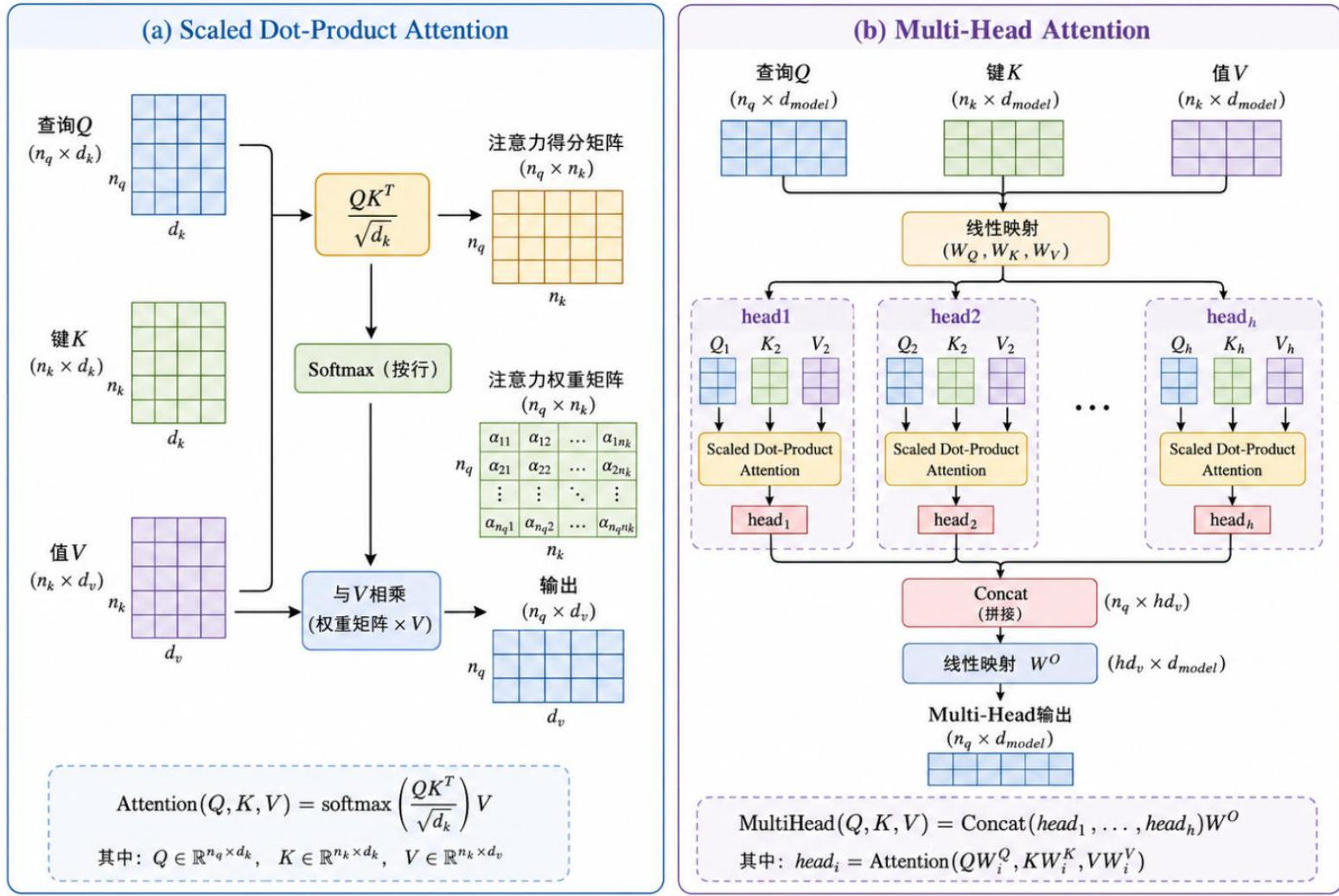


图 5 Scaled Dot-Product Attention 与 Multi-Head Attention 示意图

图 5 进一步说明了注意力计算从单头到多头的扩展过程。Scaled Dot-Product Attention 首先根据查询矩阵 Q 和键矩阵 K 计算注意力得分，经过缩放和 softmax 归一化后得到注意力权重，再利用该权重对值矩阵 V 进行加权求和，从而获得上下文表示。在此基础上，Multi-Head Attention 将这一过程扩展到多个表示子空间中并行进行，不同注意力头可以从不同角度捕捉 token 之间的关系。多个头的输出经过拼接和线性映射后，模型能够综合不同子空间的信息，获得比单头注意力更丰富的序列表示。

3.6 前馈网络、残差连接与层归一化

除注意力机制外，Transformer 中的每一层还包含位置全连接前馈网络。该网络对序列中每个位置独立作用，且所有位置共享同一组参数。其形式为

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

其中，第一层线性变换将 d_{model} 维表示扩展到更高维的隐藏空间，随后经过 ReLU 激活函数，再由第二层线性变换映射回 d_{model} 维。注意力机制主要负责不同位置之间的信息交互，而 FFN 则负责对每个位置的表示进行非线性特征变换。二者配合使用，使 Transformer 既能捕捉词间依赖，又能增强单个位置表示的非线性表达能力。

残差连接和层归一化是 Transformer 稳定训练的重要结构。对于任一子层 Sublayer(x)，其输出形式可写为

$$\text{Output} = \text{LayerNorm}(x + \text{Sublayer}(x))$$

残差连接将子层输入直接加到子层输出上，为梯度传播提供了更直接的路径，有助于缓解深层网络训练中的梯度消失问题。层归一化则对每个样本的特征维度进行标准化，使各层输出的数值分布更加稳定，从而加快训练收敛并降低模型对初始化和学习率的敏感性。本文实现中，每个 Encoder Layer 和 Decoder Layer 的注意力子层、交叉注意力子层和前馈网络子层均包含残差连接与层归一化。

3.7 Mask 机制与训练、推理过程

Transformer 在机器翻译任务中需要处理不同长度的句子，并且 Decoder 需要按自回归方式生成目标序列。因此，Mask 机制是模型实现中不可缺少的部分。本文主要使用两类 Mask：Padding Mask 和 Subsequent Mask。

Padding Mask 用于屏蔽填充位置。在同一个 batch 中，不同句子的长度通常不同，为了组成统一张量，需要使用<pad>将短句补齐。但<pad>并不对应真实 token，如果模型在注意力计算中关注这些位置，就会引入无意义信息。因此，在 Encoder 和 Decoder 的注意力计算中，需要将<pad>位置对应的注意力分数设为极小值，使 softmax 后这些位置的权重近似为 0。

Subsequent Mask 又称 Causal Mask，主要用于 Decoder 的自注意力层。训练时，虽然完整目标句已经给出，但模型在预测第 t 个词时不能看到第 t+1 及之后的真实词，否则会造成信息泄漏。Subsequent Mask 通常是一个上三角矩阵，用于屏蔽未来位置，使目标端第 t 个位置只能关注第 1 到第 t 个位置。这样可以保证训练过程与推理过程一致。

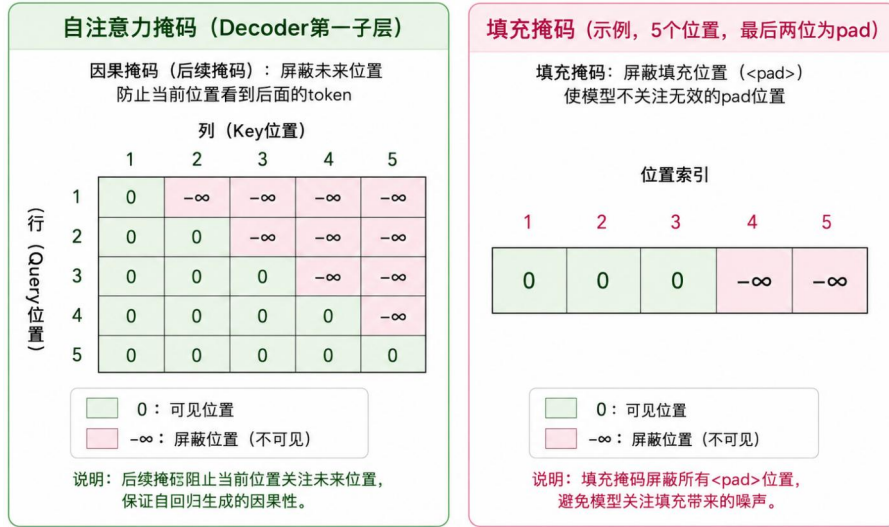


图 6 Transformer 中的 Mask 机制示意图

图 6 所示的 Mask 机制主要用于解决两个问题：一是批量训练中的无效填充位置，二是 Decoder 端可能出现的未来信息泄漏。Padding Mask 用于屏蔽由<pad>产生的填充位置，避免模型将无意义的补齐 token 纳入注意力计算；Subsequent Mask 用于 Decoder 端自注意力层，使当前位置只能关注当前及之前的目标 token，而不能提前看到未来 token。二者分别保证了批量训练的有效性和自回归生成过程的合理性，是 Transformer 机器翻译模型正常训练的重要条件。

在训练阶段，本文采用 teacher forcing 策略。为清楚说明 Decoder 端的输入与监督目标，设目标语言真实内容为

$$Y = (y_1, y_2, \dots, y_n)$$

在加入句首和句尾特殊符号后，完整目标序列可写为

$$Y^+ = (\langle \text{bos} \rangle, y_1, y_2, \dots, y_n, \langle \text{eos} \rangle)$$

训练时，Decoder 输入取完整目标序列的前 n+1 个位置，即

$$Y_{in} = (\langle \text{bos} \rangle, y_1, y_2, \dots, y_n)$$

对应的预测目标则取完整目标序列的后 n+1 个位置，即

$$Y_{label} = (y_1, y_2, \dots, y_n, \langle \text{eos} \rangle)$$

由此可以看出，Decoder 在训练时并不是直接看到完整答案后再复制输出，而是在每个位置根据<bos>和此前真实 token 预测下一个 token。例如，第一个位置根据<bos>预测 y_1 ，第二个位置根据<bos>, y_1 预测 y_2 ，最后一个位置根据已有目标句内容预测<eos>。这种右移输入和左移标签的构造方式保证了训练过程与推理阶段的自回归生成形式一致。

训练损失采用交叉熵损失，并在计算时忽略<pad>位置的贡献。设目标词表大小为 $|V|$ ，第 t 个非填充位置的真实 token 为 y_t ，模型对该位置输出的概率分布为 p_t ，则交叉熵损失可写为

$$\mathcal{L} = -\frac{1}{N} \sum_{t=1}^N \log p_t(y_t)$$

其中， N 表示参与损失计算的非<pad>token 数量。由于机器翻译任务中目标词表较大，若模型在训练中过度相信单一正确标签，可能会降低泛化能力。为缓解这一问题，本文正式实验中引入 label smoothing，平滑系数设为 0.1。该方法不会将真实标签完全视为概率 1，而是将少量概率分配给其他 token，从而使模型输出分布更加平滑，降低过拟合风险。

在推理阶段，目标端真实句子不可见，模型需要按照自回归方式逐步生成译文。具体而言，Decoder 首先接收<bos>作为初始输入，经过模型计算得到第一个目标 token 的概率分布，并选择概率最大的 token 作为当前输出；随后将该 token 追加到 Decoder 输入序列中，继续预测下一个 token。上述过程不断重复，直到模型生成<eos>或达到最大序列长度 max_len。本文推理阶段采用贪心解码策略，其形式可表示为

$$\hat{y}_t = \arg \max_{w \in V} P(w | \hat{y}_1, \hat{y}_2, \dots, \hat{y}_{t-1}, X)$$

其中， $\hat{y}_t$ 表示模型在第 t 步生成的 token， V 表示目标语言词表。贪心解码每一步都选择当前概率最大的 token，计算简单、速度较快，适合本文以 Transformer 结构复现和位置编码对比为主的实验任务。不过，该方法也存在一定局限，即每一步只保留单个最优选择，无法像 beam search 那样同时探索多条候选翻译路径。因此，在后续翻译样例分析中，本文主要将生成结果作为模型基本翻译能力的定性参考，而不将其等同于完整机器翻译系统的最优输出。

3.8 本文比较的五种位置编码

位置编码形式是本文实验的核心变量。由于自注意力机制本身主要根据 token 表示之间的相似度分配权重，而不直接包含 token 的顺序信息，因此模型需要通过位置编码获得与序列顺序相关的额外表示。为研究不同位置表示形式对 Transformer 翻译性能的影响，本文共设置五组位置编码实验，分别为 none、absolute、sinusoidal、learned 和 gated_mix。五组实验除位置编码方式不同外，其

余模型结构、训练数据划分和超参数设置均保持一致。

第一种为无位置编码，即 **none** 组。该组不向词嵌入中加入任何显式位置信息，只使用 **token embedding** 作为模型输入。该实验相当于位置编码消融实验，用于观察在完全缺少显式位置信息的情况下，Transformer 是否仍能依赖词语共现关系和上下文模式完成一定程度的翻译任务。**none** 组的结果可以作为判断位置编码必要性的基础对照。

第二种为简单绝对位置编码，即 **absolute** 组。按照作业要求，本文设计了一种固定的简单绝对位置编码，其思想是直接根据 **token** 在序列中的位置和向量维度构造确定性位置向量。其定义为

$$P_{abs}(pos, j) = \left(2 \frac{pos}{L_{max} - 1} - 1\right) \left(2 \frac{j}{d_{model} - 1} - 1\right)$$

其中， pos 表示 **token** 在序列中的绝对位置， j 表示位置编码向量中的维度索引， L_{max} 表示最大序列长度。该编码将位置索引和维度索引均归一化到 $[-1, 1]$ 区间，不引入任何可训练参数。它的优点是形式简单、实现方便，可以为模型提供最基本的绝对位置信息；但其表达形式仅为线性变化，缺乏多尺度频率结构，难以显式刻画不同位置之间的相对距离关系。

第三种为原论文正弦位置编码，即 **sinusoidal** 组。该方法是《Attention Is All You Need》中提出的固定位置编码方式，通过不同频率的正弦和余弦函数为不同位置构造向量表示^[1]。其定义为

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right), PE(pos, 2i + 1) = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

其中， pos 表示位置， i 表示维度对的索引。正弦位置编码的不同维度对应不同频率的周期函数，低频维度能够描述较长尺度的位置变化，高频维度则能够区分较近位置之间的差异。相比简单绝对位置编码，正弦编码不仅能够区分不同绝对位置，还能够通过周期函数的组合为模型提供相对位置关系的表达基础。因此，它通常被认为是一种更适合序列建模的固定位置表示。

第四种为可学习绝对位置编码，即 **learned** 组。该方法为每个位置设置一个可训练的位置嵌入向量，并在训练过程中与模型其他参数一起通过反向传播更新。若位置索引为 pos ，则其位置编码可表示为

$$P_{learned}(pos) = E_{pos}[pos]$$

其中， E_{pos} 表示可学习的位置嵌入矩阵。与固定编码相比，**learned** 位置编码

的优势在于灵活性更强，能够根据当前训练数据自动调整每个位置的向量表示，从而学习更适合具体任务的位置模式。但其不足也较为明显：当测试序列长度超出训练时设定的最大长度时，可学习位置嵌入无法自然外推到未见过的位置。因此，**learned** 编码更适合训练和测试长度范围相对固定的任务。

第五种为门控混合位置编码，即 **gated_mix** 组。这是本文在作业要求基础上加入的一项轻量改进尝试。其基本思想是将原论文正弦位置编码和简单绝对位置编码进行可学习融合，使模型在训练过程中自动调节两类位置信息的贡献比例。其形式为

$$P_{mix} = \sigma(\alpha)P_{sin} + \sigma(\beta)P_{abs}$$

其中， P_{sin} 表示正弦位置编码， P_{abs} 表示简单绝对位置编码， α 和 β 为可学习标量参数， $\sigma(\cdot)$ 为 sigmoid 函数。经过 sigmoid 函数约束后，门控系数位于 0 到 1 之间，可以解释为模型对对应位置编码的利用程度。训练完成后，本文记录 $\sigma(\alpha)$ 和 $\sigma(\beta)$ 的数值，用于分析模型是否更倾向于利用正弦编码或简单绝对编码。

需要说明的是，**gated_mix** 并不是对 Transformer 主体结构的改变，也不是一种复杂的新模型，而是在位置编码层面进行的轻量融合尝试。它几乎不增加模型参数量，其目的在于检验两类固定位置编码之间是否存在互补性。如果模型能够通过门控机制获得优于单一固定编码的效果，则说明不同位置表示可能提供了互补信息；若效果没有明显提升，则说明简单标量门控可能不足以有效融合两类编码。

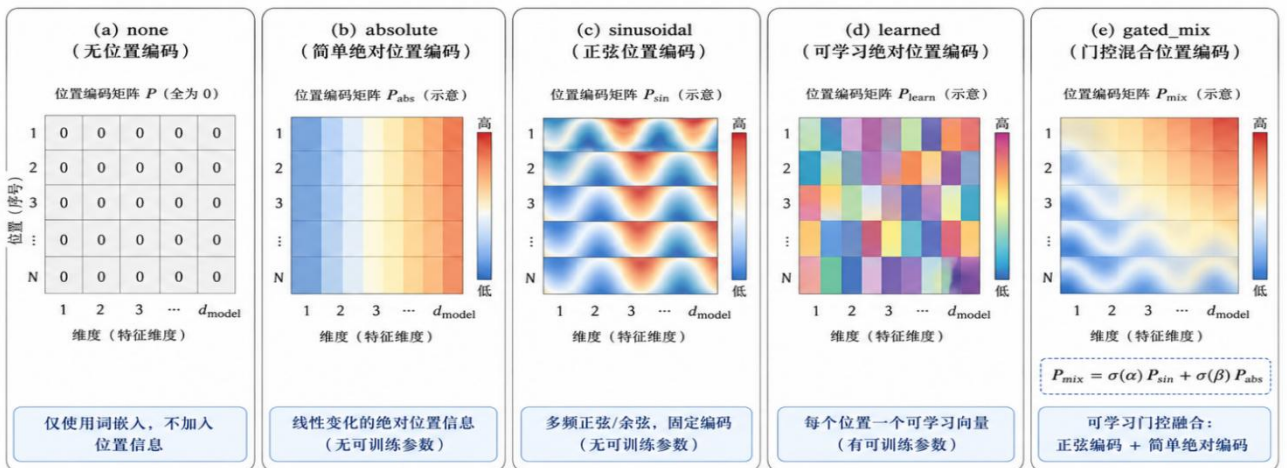


图 7 不同位置编码方式示意图

图 7 对本文实验中使用的五种位置编码方式进行了概括。**none** 组完全不加入显式位置信息，用于观察模型在缺少位置编码时的表现；**absolute** 组提供简单

线性绝对位置信息；sinusoidal 组采用原论文中的多频正弦位置编码；learned 组通过训练直接学习每个位置的向量表示；gated_mix 组则尝试对正弦编码和简单绝对编码进行可学习融合。通过这些方法的对比，可以更清楚地分析不同位置表示形式对 Transformer 翻译性能的影响。

综上，本文在模型方法部分不仅复现了经典 Transformer Encoder-Decoder 框架，也围绕位置编码模块构造了从消融实验到固定编码、可学习编码和门控混合编码的完整对比体系。后续实验将以位置编码类型作为唯一变量，在相同数据划分、模型规模和训练参数下比较五种方法的训练过程、测试集准确率、句长分桶表现以及翻译样例，从而进一步分析位置编码在 Transformer 机器翻译任务中的实际作用。

四、实验设计与评价方法

4.1 实验目的与对照思路

在完成 Transformer Encoder-Decoder 结构复现的基础上，本文进一步围绕位置编码模块设计对比实验。实验的主要目的不是单纯追求机器翻译任务上的最高准确率，而是通过控制变量的方式分析不同位置编码形式对 Transformer 翻译性能的影响。由于自注意力机制本身不直接包含序列顺序信息，位置编码是否加入、如何加入以及采用何种形式，都会影响模型对句子结构和长距离依赖关系的建模能力。

为保证实验结论具有可比性，本文将位置编码类型设置为唯一主要变量，分别设计 none、absolute、sinusoidal、learned 和 gated_mix 五组实验。其中，none 组用于观察模型在不加入显式位置信息时的表现；absolute 组用于检验简单绝对位置信息是否足以支撑翻译任务；sinusoidal 组用于复现原论文中的固定正弦位置编码；learned 组用于考察可学习位置表示在当前数据规模下的效果；gated_mix 组则作为本文的轻量改进尝试，用于分析正弦编码与简单绝对编码之间是否存在互补性。

除位置编码类型外，五组实验在数据划分、模型规模、训练轮数、优化器、学习率、batch size 和评价方式上均保持一致。这样可以尽量避免其他因素对实验结果造成干扰，使不同实验组之间的差异主要反映位置编码方式本身的影响。

4.2 实验配置与训练参数

本文正式实验使用 100,000 对德英平行句作为样本，并按照 80%、10%、10% 的比例划分为训练集、验证集和测试集。模型采用 3 层 Encoder 和 3 层 Decoder，隐藏维度 d_{model} 设置为 256，多头注意力头数为 8，前馈网络中间维度为 1024。训练过程中使用 Adam 优化器^[4]，学习率设为 0.0003，batch size 设为 128，总训练轮数为 20epoch。为提高训练稳定性并减少过拟合风险，实验中使用 dropout 和 label smoothing，其中 label smoothing 系数为 0.1。

本文正式实验主要参数如表 1 所示。

表 1 正式实验主要参数设置

参数	设置
任务方向	德语→英语
实验样本数	100,000
训练集/验证集/测试集	79,994 / 9,999 / 10,000
源语言词表大小	30,000
目标语言词表大小	22,259
max_len	30
d_model	256
n_heads	8
num_layers	3
dim_feedforward	1024
batch_size	128
epochs	20
learning_rate	0.0003
dropout	0.1
label_smoothing	0.1
optimizer	Adam
device	NVIDIA GeForce RTX 4060 Laptop GPU
模型参数量	约 24.6M
评价指标	token accuracy、BLEU

从表 1 可以看出，本文所采用的模型规模明显小于原论文 Transformer base 配置，但已经能够完整体现 Encoder-Decoder、自注意力、Cross-Attention、位置编码和自回归解码等关键结构。考虑到课程作业的算力条件和实验目标，本文没有使用大规模预训练模型，也没有调用 HuggingFace 等高级封装库，而是通过纯 PyTorch 实现模型主体结构，以便更清楚地观察不同位置编码方法对模型训练和翻译结果的影响。

4.3 五组位置编码实验设置

为了系统比较位置编码对翻译任务的影响，本文设置了五组并行实验。五组实验的模型主体结构完全一致，仅在位置编码层进行替换。具体实验设置如表 2 所示。

表 2 五组位置编码对照实验设置

实验组	位置编码方式	是否可训练	作用
none	不加入位置编码	否	作为位置编码消融对照，观察模型在缺少显式位置信息时的表现
absolute	简单绝对位置编码	否	检验简单线性绝对位置信息对翻译任务的作用
sinusoidal	原论文正弦位置编码	否	复现经典 Transformer 中的固定多频位置编码方法
learned	可学习绝对位置编码	是	检验模型能否从训练数据中学习更适合任务的位置表示
gated_mix	门控混合位置编码	部分可训练	通过可学习门控融合正弦编码和简单绝对编码，检验二者是否具有互补性

表 2 中的 none 组相当于位置编码消融实验，用于判断显式位置信息是否必要；absolute 组提供最简单的固定绝对位置表示，可用于和原论文正弦位置编码进行直接对比；sinusoidal 组对应《Attention Is All You Need》中的标准位置编码方法，是本文复现实验的重要组成部分；learned 组将位置向量设为可训练参数，用于检验模型能否从数据中学习更适合当前翻译任务的位置表示；gated_mix 组则在正弦位置编码和简单绝对位置编码之间引入可学习门控，作为对位置编码融合方式的轻量探索。

需要说明的是，本文在第四部分不再重复五种位置编码的数学定义，因为其公式与原理已经在第三部分模型方法中给出。这里更关注实验分组的作用和公平对比关系。通过这种设置，后续结果分析可以围绕“无位置编码、简单位置编码、经典正弦编码、可学习编码和融合编码”五类方法展开，使结论更加清晰。

4.4 训练流程

每组实验均训练 20epoch。训练过程中，每个 epoch 结束后记录训练集损失、验证集损失以及验证集上的 token accuracy。训练集损失用于观察模型对训练数据的拟合情况，验证集损失用于监控模型泛化表现并选择最佳模型。为避免直接利用测试集调节模型参数，本文以验证集 valid_loss 最低的 epoch 作为 best_epoch，并保存该 epoch 对应的 checkpoint。

最终性能评价时，本文加载每组实验中验证集损失最低的 checkpoint，在测试集上计算 token_acc、BLEU、short_acc、medium_acc 和 long_acc。因此，本文后续结果表中的 valid_loss 来自验证集，而最终 token_acc 及句长分桶准确率均来自测试集。这种处理方式能够保证模型选择和最终评价之间相互分离，使实验结论更加规范。

训练日志中保存了每组模型在不同 epoch 下的损失和准确率变化，便于后续绘制训练曲线并分析收敛过程。最终汇总结果则用于比较五种位置编码方法在相同训练条件下的整体性能差异。

4.5 评价指标与句长分桶

本文采用 token accuracy 和 BLEU 作为主要定量评价指标。其中，token accuracy 用于衡量模型生成结果与参考译文在 token 位置上的匹配程度，计算方式较为直观，适合比较五组实验之间的相对性能差异；BLEU 则从 n-gram 重合角度评价生成译文与参考译文之间的整体相似性，是机器翻译任务中常用的补充评价指标^[6]。需要注意的是，token accuracy 更关注逐位置 token 匹配，而 BLEU 更关注整句层面的短语重合程度，二者评价角度并不完全相同。因此，本文将 token accuracy 作为位置编码对比的主要指标，同时补充 BLEU 结果，并结合翻译样例进行定性分析。

除整体 token accuracy 和 BLEU 外，本文还根据参考译文长度对测试集进行分桶，分别计算短句、中句和长句上的准确率。其中，短句定义为长度不超过 6

个 token 的句子，中句定义为长度为 7 到 12 个 token 的句子，长句定义为长度超过 12 个 token 的句子。这样划分的原因在于，不同长度句子对位置编码的依赖程度不同。短句结构相对简单，模型更容易依靠词语共现和局部上下文完成翻译；长句则通常包含更复杂的句法结构和更长距离的依赖关系，对模型的位置建模能力提出更高要求。

因此，本文后续不仅比较五组实验的整体 token accuracy 和 BLEU，还会进一步分析 short_acc、medium_acc 和 long_acc。通过句长分桶结果，可以更细致地观察不同位置编码方法在短句、中句和长句上的表现差异，从而判断位置编码的作用是否主要体现在更复杂的序列结构建模中。

五、实验结果与分析

5.1 训练过程分析

为观察五种位置编码方法在训练过程中的收敛情况，本文记录了每组实验在 20 个 epoch 内的训练集损失和验证集损失变化情况。图 8 展示了五组模型的训练损失与验证损失曲线。

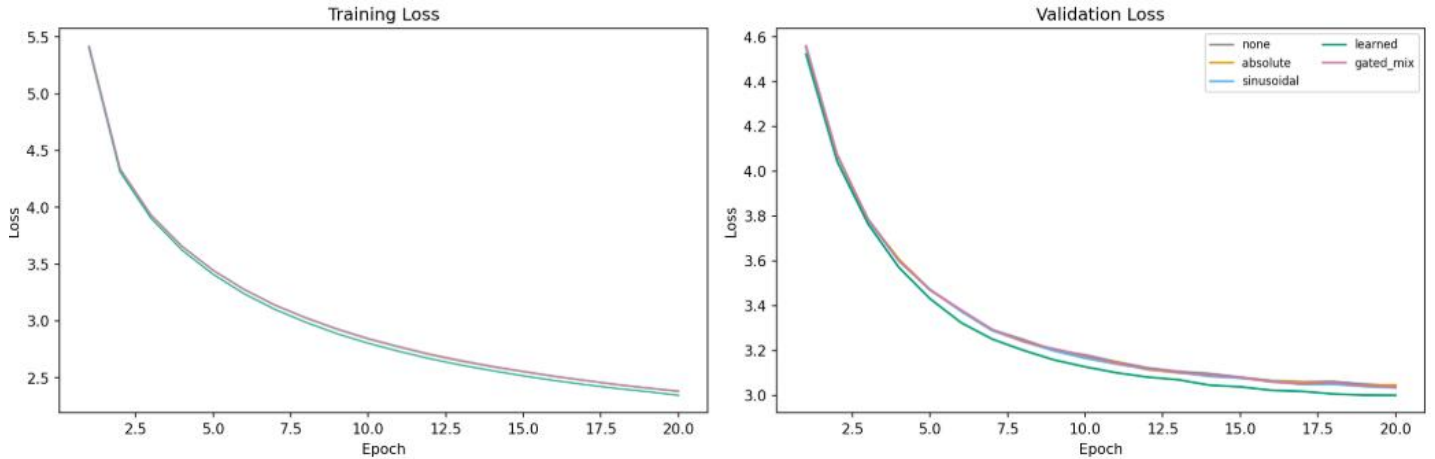


图 8 五种位置编码方法的训练与验证损失曲线

从图 8 可以看出，五组实验的训练损失和验证损失整体均随 epoch 增加而持续下降，说明模型在不同位置编码设置下均能够正常学习德英翻译任务。训练初期，各组曲线差异较小，表明在模型尚未充分收敛时，位置编码方式对整体损失的影响并不明显。随着训练轮数增加，不同方法之间的差异逐渐显现，其中 learned 位置编码的验证损失下降更充分，最终在五组方法中取得最低的 valid_loss。

具体来看，learned 组在第 20 个 epoch 时验证集损失降至 3.0002，低于

sinusoidal 组的 3.0336、gated_mix 组的 3.0372、absolute 组的 3.0421 和 none 组的 3.0440。这说明在当前数据规模和训练配置下，可学习位置编码能够通过训练过程逐步调整位置向量，使其更适应德英翻译任务的数据分布。相比之下，sinusoidal、absolute 和 none 等固定或缺失位置编码的方法虽然也能正常收敛，但最终验证损失略高，说明其位置表示能力或序列结构表达能力存在一定限制。

此外，从曲线变化趋势来看，前 10 个 epoch 是模型损失下降较快的阶段，之后下降速度逐渐减缓。到 20epoch 时，各组曲线已经趋于平稳，但 learned 组仍保持较好的下降趋势。这一现象说明当前训练轮数已经能够形成较稳定的实验比较，同时也表明若进一步扩大训练轮数，模型仍可能获得少量提升。不过，本文的实验重点是比较不同位置编码方式的相对差异，而不是追求极限翻译性能，因此 20epoch 的训练结果已经能够支撑后续分析。

5.2 五种位置编码整体性能比较

为进一步比较不同位置编码方式的最终性能，本文加载每组实验中验证集损失最低的 checkpoint，并在测试集上计算整体 token accuracy、BLEU 及不同句长分桶准确率。表 3 给出了五种位置编码方法的验证集损失、测试集 BLEU、整体 token accuracy 及不同句长分桶准确率对比结果。

表 3 五种位置编码方法的验证集损失、BLEU 与测试集准确率对比

pe_type	best_epoch	valid_loss (验证集)	BLEU (测试集)	token_acc (测试集)	short_acc (测试集)	medium_acc (测试集)	long_acc (测试集)
none	20	3.0440	36.22	0.4684	0.5429	0.4265	0.1893
absolute	19	3.0421	35.18	0.4649	0.5460	0.4186	0.1795
sinusoidal	20	3.0336	36.03	0.4679	0.5416	0.4268	0.1959
learned	20	3.0002	36.93	0.4979	0.5715	0.4586	0.2208
gated_mix	20	3.0372	35.87	0.4680	0.5462	0.4235	0.1905

从表 3 可以看出，learned 位置编码在所有主要指标上均取得最优结果，其测试集 token_acc 达到 0.4979，BLEU 达到 36.93，明显高于其他四组方法。同时，learned 组在 short_acc、medium_acc 和 long_acc 上也分别达到 0.5715、0.4586 和

0.2208, 说明其优势并不只体现在某一类句子上, 而是对不同长度的句子均有稳定提升。这一结果表明, 在本文所使用的数据规模和 `max_len=30` 的长度范围内, 可学习位置编码能够根据训练数据自适应地学习位置表示, 从而比固定位置编码更适合当前翻译任务。

从 BLEU 结果看, `learned` 组同样取得最高分数, 与其 `token_acc` 和句长分桶指标上的最优表现基本一致。这说明 `learned` 位置编码的优势不仅体现在逐位置 `token` 匹配上, 也体现在整句翻译质量上。`absolute` 组 BLEU 最低, 为 35.18, 与其整体 `token_acc` 和 `long_acc` 偏低的结果一致, 进一步说明简单线性位置编码对复杂序列结构的表达能力较弱。

需要注意的是, `none` 组 BLEU 为 36.22, 略高于 `sinusoidal` 组的 36.03, 但 `sinusoidal` 组在 `long_acc` 上优于 `none`。这说明 BLEU 和 `token accuracy` 关注角度不同, 整体测试集中短句占比较高时, BLEU 可能更受常见短句翻译效果影响; 而句长分桶结果更能反映位置编码在长句结构建模中的差异。

`sinusoidal` 位置编码在长句分桶中表现较好。虽然其整体 `token_acc` 和 BLEU 与 `none`、`gated_mix` 接近, 但其 `long_acc` 达到 0.1959, 高于 `none` 的 0.1893、`absolute` 的 0.1795 和 `gated_mix` 的 0.1905。这说明原论文中的多频正弦位置编码在长句场景下具有一定优势。长句往往包含更复杂的句法结构和更长距离依赖关系, 对位置编码质量更加敏感。`sinusoidal` 编码通过不同频率的正弦和余弦函数构造位置表示, 能够为模型提供比简单线性编码更丰富的位置关系信息, 因此在长句分桶中优于 `absolute` 组。

`absolute` 位置编码整体表现最弱, 其 `token_acc` 为 0.4649, BLEU 为 35.18, `long_acc` 仅为 0.1795。虽然 `absolute` 组向模型提供了显式绝对位置信息, 但该编码形式过于简单, 只通过位置索引和维度索引的线性归一化构造位置向量, 缺少多尺度周期结构, 也难以表达相对位置关系。因此, `absolute` 组虽然比完全没有位置信息更加符合 Transformer 设计思想, 但并未在实验中体现出明显优势, 反而在长句上低于 `none` 组。这表明“加入位置编码”并不必然带来性能提升, 位置编码本身的表达形式同样关键。

`none` 组没有加入显式位置编码, 但其 `token_acc` 仍达到 0.4684, BLEU 达到 36.22, 与 `sinusoidal` 和 `gated_mix` 接近。这一现象说明, 在当前数据中大量句子

较短、句式较简单的情况下，模型可以依靠 token embedding、上下文共现和目标端语言模式完成一定程度的翻译。也就是说，位置编码并不是模型能否学习到基本翻译关系的唯一因素。但是，none 组在 long_acc 上低于 sinusoidal 和 learned，说明缺少显式位置信息时，模型在更复杂的长句结构中仍然存在不足。

gated_mix 组的 token_acc 为 0.4680，BLEU 为 35.87，整体表现与 none 和 sinusoidal 接近，但没有超过 learned。训练结束后，gated_mix 中两个门控值分别为 $\sigma(\alpha)=0.9462$ 和 $\sigma(\beta)=0.9017$ ，说明模型倾向于同时保留正弦位置编码和简单绝对位置编码的信息，而不是完全舍弃其中一种。虽然该结果表明门控参数确实参与了学习过程，但从最终指标看，简单的标量门控融合并没有带来明显增益。这可能是因为正弦编码和简单绝对编码之间的互补信息有限，也可能是因为标量门控只能整体调节两类编码的比例，表达能力不如按维度或按位置变化的细粒度门控。

5.3 句长分桶结果分析

为分析位置编码在不同长度句子上的作用差异，本文进一步比较五组方法在短句、中句和长句三个分桶上的测试集 token accuracy。图 9 展示了不同句长分桶下五种位置编码方法的准确率对比。

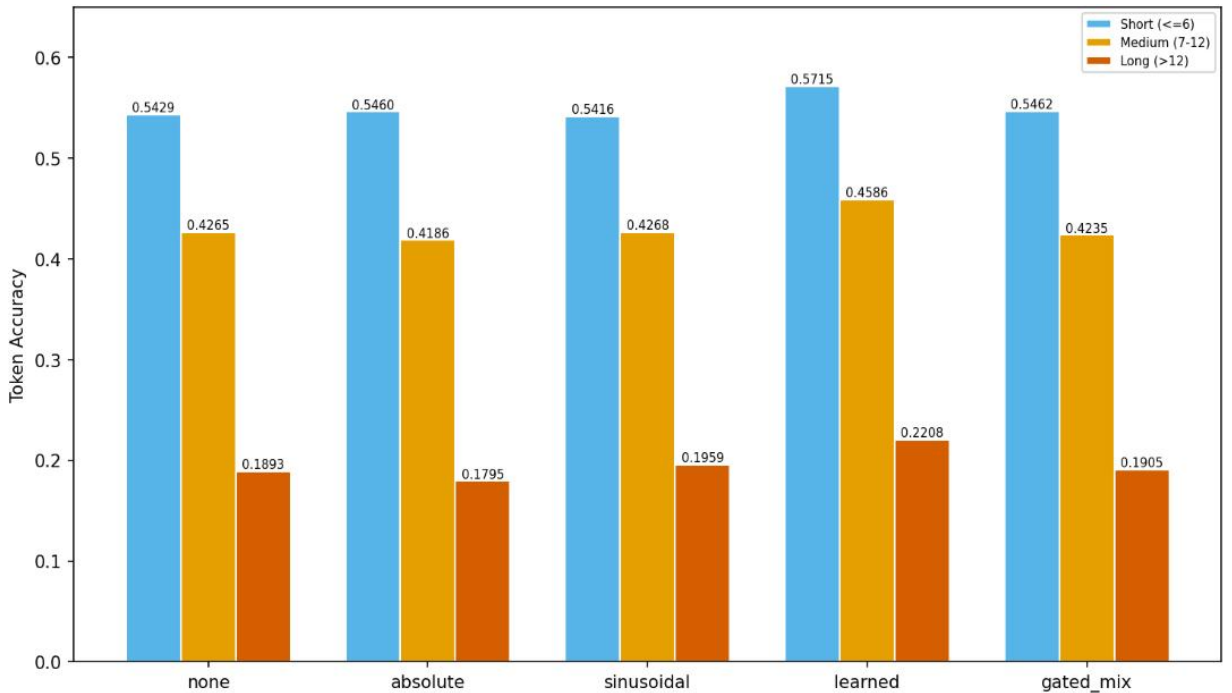


图 9 不同句长分桶下的测试集 token accuracy 对比

从图 9 可以看出，所有方法均表现出明显的句长敏感性：短句准确率最高，

中句次之，长句最低。以 `learned` 组为例，`short_acc` 为 0.5715，`medium_acc` 下降为 0.4586，`long_acc` 进一步下降为 0.2208。这说明随着句子长度增加，模型需要处理的 `token` 数量、句法结构和长距离依赖关系都显著增加，翻译难度也随之上升。对于本文这种从零训练的小型 Transformer 模型而言，长句翻译仍然是主要难点。

不同位置编码方法之间的差异在长句上更容易体现。短句通常结构简单，局部词语共现信息已经能够支持模型生成较合理的翻译，因此不同位置编码方法之间的差距相对较小。而长句包含更多修饰成分、从句结构或长距离依赖，模型需要更准确地理解 `token` 在序列中的相对位置关系，因此更依赖有效的位置编码。从结果看，`learned` 组在长句上达到 0.2208，明显高于其他方法；`sinusoidal` 组在固定编码中取得最高的 `long_acc`，为 0.1959；`absolute` 组的 `long_acc` 仅为 0.1795，说明简单线性位置编码在复杂序列建模中效果有限。

值得注意的是，`absolute` 组在 `short_acc` 上达到 0.5460，略高于 `none` 和 `sinusoidal`，但在 `medium_acc` 和 `long_acc` 上均表现较弱。这说明简单绝对位置编码可能在短句中提供了一定直接位置信息，但当句子变长、位置关系更复杂时，其线性表达形式不足以支撑更细致的序列结构建模。相比之下，`sinusoidal` 组虽然整体 `token_acc` 并非最高，但在长句上的相对优势说明原论文正弦位置编码的多频结构确实更适合处理较长序列。

`gated_mix` 组在三个分桶上的结果介于 `none` 和 `sinusoidal` 附近，并未表现出显著提升。结合其较高的门控值可以推测，模型虽然同时引入了正弦编码和简单绝对编码，但简单标量加权可能无法有效区分不同位置、不同维度或不同句长场景下两类编码的作用。因此，若后续继续改进 `gated_mix`，可以考虑使用向量级门控或按维度门控，使模型能够更细致地选择不同类型的位置信息。

5.4 翻译样例分析

除定量指标外，本文还选取部分测试集样例，对五种位置编码方法生成的译文进行定性比较。需要说明的是，本文采用的评价指标包括 `token accuracy` 和 `BLEU`，而自然语言翻译存在多种合理表达方式，因此某些译文即使与参考译文 `token` 不完全一致，也可能在语义上基本正确。翻译样例分析的目的不是替代定量评价，而是帮助观察不同模型在具体句子上的生成行为。

表 4 展示了部分具有代表性的翻译样例，其中既包含短句和中句，也包含长句样例，以便观察不同句长下模型生成结果的差异。

表 4 五种位置编码方法的部分翻译样例对比

句长类型	源句	参考译文	none	absolute	sinusoidal	learned	gated_mix
短句	haben sie gestern abend angerufen?	did you call last night?	did you call last night?	did you phone last night?	did you call last night?	did you call last night?	did you call last night?
中句	ich stimme tom hundertprozentig zu.	i agree with tom a hundred percent.	i agree with tom a hundred per cake.	i agree with tom a hundred per father.	i agree with tom agree on his phone.	i agree with tom a hundred times.	i agree with tom a hundred times.
长句	nach einer suche von oben bis unten durch das ganze haus fand tom die <unk> die er verloren zu haben glaubte, wieder.	tom found the wallet he thought he'd lost after searching the house from top to bottom.	tom thought the police were looking for a whole hour that he was looking for the number of all the books that had been looking for the way to	tom thought the whole house was looking for the whole body to find out of the whole story.	tom thought that he had been looking for a whole house by the whole house before the whole city takes to find out of his house all over the	tom thought he had been looking for a whole house by the whole house to pass the entire time.	tom found the house all the house that he thought that was looking for the people who had lost the problem from looking for.
长句	ich halte es für unwahrscheinlich, daß tom das machen kann.	i think that it's unlikely that tom will be able to do that.	i think it's unlikely that tom can do that for you.	i think it's unlikely tom can do that for you.	i think it's unlikely that tom will be able to do that for you.	i think it's unlikely tom can do that for you.	i think it's unlikely that tom can do that.

从表 4 可以看出，在短句样例“haben sie gestern abend angerufen?”中，除 absolute 将 call 替换为 phone 外，其余方法均能生成与参考译文完全一致或高度接近的结果。这说明对于结构简单、上下文依赖较短的句子，模型主要依靠 token embedding 和常见表达模式即可完成较准确翻译，位置编码方式对最终输出的影响相对有限。

在中句样例“ich stimme tom hundertprozentig zu.”中，五种方法均保留了“I agree with Tom”的主要语义，但在“hundred percent”这一短语上出现不同程度偏差。其中 none 和 absolute 分别生成“a hundred per cake”和“a hundred per father”，sinusoidal 生成的结果也出现重复和语义偏移，learned 和 gated_mix 虽然没有完全匹配参考译文，但整体含义更接近原句。这说明随着句子长度和表达结构增加，模型不仅需要识别局部词义，还需要更稳定地处理短语搭配和句法关系。

长句样例更明显地体现了当前模型的不足。对于“nach einer suche von oben bis unten durch das ganze haus...”这一复杂长句，五组方法均未能完整恢复“从

上到下搜索整栋房子后找回钱包”的语义结构，生成结果中普遍存在重复、误译和成分缺失。这与前文 `long_acc` 整体较低的结果一致，说明从零训练的小型 Transformer 在处理复杂长距离依赖时仍有明显局限。相比之下，在“`ich halte es für unwahrscheinlich...`”这一结构相对清晰的长句中，`sinusoidal` 和 `gated_mix` 生成结果更接近参考译文，`none`、`absolute` 和 `learned` 也能保留主要语义，但存在“`for you`”等额外成分。这说明长句并非完全无法翻译，但句法结构越复杂，模型越容易出现局部语义偏差。

总体来看，翻译样例与定量指标基本一致：短句中不同位置编码方法差距较小，而中句和长句更容易暴露模型在顺序建模和结构理解上的不足。`learned` 在整体 `token accuracy` 和 BLEU 上表现最优，但并非每个单独样例都完全正确；`sinusoidal` 在部分长句中表现出较稳定的结构保持能力；`absolute` 和 `none` 更容易出现重复、替换或语义偏移现象。因此，翻译样例可以作为定量指标之外的补充分析，帮助观察不同位置编码方式在具体生成过程中的差异。

5.5 与前期轻量实验的对比

为了验证扩大数据规模和模型容量对实验结果的影响，本文在正式实验之前进行了两轮较小规模训练。第一轮使用 8K 样本、5epoch 和 `d_model=128`，主要用于验证代码和数据管线；第二轮使用 50K 样本、12epoch 和 `d_model=256`，用于观察 GPU 训练速度和中等规模训练效果。表 5 以 `learned` 位置编码为例，对比三种实验规模下的结果。

表 5 不同实验规模下 `learned` 位置编码的结果对比

实验版本	样本数	epoch	d_model	token_acc	short_acc	medium_acc	long_acc
前期轻量实验	8K	5	128	0.1732	0.2083	0.1490	0.0722
中等规模实验	50K	12	256	0.4110	0.4757	0.3715	0.2061
正式实验	100K	20	256	0.4979	0.5715	0.4586	0.2208

从表 5 可以看出，随着训练样本数量、训练轮数和模型容量增加，`learned` 组的各项指标均得到明显提升。整体 `token_acc` 从 0.1732 提升至 0.4979，约为原来的 2.87 倍；`long_acc` 从 0.0722 提升至 0.2208，约为原来的 3.06 倍。这说明在从零训练 Transformer 的场景下，数据规模和训练充分程度对翻译质量具有重要影响。较小规模实验虽然能够验证模型结构和训练流程是否正确，但不足以充分

体现模型的翻译能力和不同位置编码方法之间的差异。

同时也可以看到，从 50K 样本提升到 100K 样本后，模型仍然获得了进一步改进，但提升幅度小于从 8K 到 50K 阶段。这说明在当前模型结构和简单空格分词条件下，继续增加训练数据仍有帮助，但边际收益开始下降。若希望进一步提高翻译质量，可能需要同时改进分词方式、模型规模、解码策略和评价指标，而不仅仅是增加样本数量或训练轮数。

综上，本文的正式实验相较前期轻量实验更能反映不同位置编码方式的真实差异。前期实验主要承担调试和验证作用，而正式实验则作为本文结论分析的主要依据。通过多规模训练对比可以看出，Transformer 模型在数据规模和训练轮数增加后能够明显提升翻译能力，但位置编码方法仍会在相同训练条件下造成稳定的性能差异。

六、结论

本文围绕德语到英语机器翻译任务，使用纯 PyTorch 从零复现了 Transformer Encoder-Decoder 结构，并在此基础上针对位置编码模块开展了对比实验。本文没有使用 HuggingFace 框架或预训练模型，而是完整实现了词嵌入、位置编码、多头注意力、Encoder、Decoder、Mask 机制、teacher forcing 训练和贪心解码流程。通过这一过程，本文不仅完成了经典 Transformer 架构的复现，也进一步分析了位置编码在序列建模中的作用。

在实验设计上，本文将位置编码类型作为唯一主要变量，分别设置 none、absolute、sinusoidal、learned 和 gated_mix 五组实验。五组实验除位置编码方式不同外，其余数据划分、模型规模、训练轮数、优化器和评价方式均保持一致，从而保证实验结果具有较好的可比性。实验结果表明，learned 位置编码在本文正式实验中取得了最优表现，其测试集 token_acc 达到 0.4979，BLEU 达到 36.93，并且在短句、中句和长句三个分桶上均优于其他方法。这说明在当前 100K 样本、max_len=30 和 20epoch 训练条件下，可学习位置编码能够从数据中学习到更适合当前德英翻译任务的位置表示。

与 learned 位置编码相比，原论文中的 sinusoidal 位置编码虽然整体 token_acc 和 BLEU 并非最高，但在长句分桶上优于简单绝对位置编码，并且也高于 none

和 `gated_mix`。该结果说明，多频率正弦和余弦函数构造的位置表示在长距离依赖建模中具有一定优势。相比之下，`absolute` 位置编码虽然为模型提供了显式位置信息，但由于其形式过于简单，缺少多尺度变化和相对位置表达能力，因此在整体指标和长句指标上均未表现出优势。这表明，位置编码的作用不仅在于“是否加入位置信息”，还取决于位置信息本身的表示质量。

`none` 组在不加入显式位置编码的情况下仍取得了一定翻译效果，说明模型可以依靠 `token embedding`、上下文共现关系和目标端语言模式学习到部分翻译规律。但是，`none` 组在 `long_acc` 上低于 `learned` 和 `sinusoidal`，说明当句子长度增加、结构关系变复杂时，缺少显式位置信息会限制模型对序列结构的理解能力。因此，位置编码对于 `Transformer` 并非只是形式上的附加模块，而是在较复杂序列建模中具有实际意义。

本文提出的 `gated_mix` 位置编码是一种轻量改进尝试，其核心思想是通过可学习门控融合正弦位置编码和简单绝对位置编码。实验结果显示，`gated_mix` 的门控值分别达到 0.9462 和 0.9017，说明模型确实倾向于同时保留两类位置信息。然而，其最终性能并未超过 `learned`，并且在 BLEU 和 `long_acc` 上也没有明显超过 `sinusoidal`。这说明简单的标量级门控虽然能够反映模型对不同编码的利用程度，但表达能力仍然有限，难以充分挖掘两类位置编码之间的互补关系。后续可以尝试向量级门控、按维度门控或基于注意力的位置编码融合方式。

从句长分桶结果看，所有模型都表现出明显的句长敏感性，即短句准确率最高，中句次之，长句最低。这一现象说明，从零训练的小型 `Transformer` 已经能够较好地处理常见短句和中等长度句子，但在长句、复杂结构和细粒度语义表达上仍存在不足。结合翻译样例可以看出，模型在部分短句上已经能够生成较准确译文，但在涉及复杂句式、长距离依赖或细粒度语义时仍可能出现偏差。这表明本文模型已经具备基本翻译能力，但距离高质量机器翻译系统仍有明显差距。

总体来看，本文实验验证了 `Transformer Encoder-Decoder` 结构在德英翻译任务中的基本有效性，也说明位置编码方式会对模型性能产生稳定影响。在当前实验条件下，可学习位置编码整体最优，原论文正弦位置编码在长句分桶上体现出相对优势，简单绝对位置编码表现相对有限，门控混合位置编码虽然体现了一定自适应能力，但仍需要更精细的融合机制。后续研究可从以下几个方向继续改进：

首先，可采用 BPE 等子词分词方法，以缓解空格分词带来的低频词和德语复合词问题；其次，可进一步引入 beam search 等更优解码策略；最后，还可以分析位置编码相似度矩阵或不同注意力头对位置信息的利用情况，从而更深入地理解 Transformer 中的位置表示机制。

参考文献

- [1] Vaswani A, Shazeer N, Parmar N, et al. Attention Is All You Need. Advances in Neural Information Processing Systems, 2017.
- [2] Bahdanau D, Cho K, Bengio Y. Neural Machine Translation by Jointly Learning to Align and Translate. International Conference on Learning Representations, 2015.
- [3] Goodfellow I, Bengio Y, Courville A. Deep Learning. MIT Press, 2016.
- [4] Kingma D P, Ba J. Adam: A Method for Stochastic Optimization. International Conference on Learning Representations, 2015.
- [5] Tatoeba Project. Tatoeba Sentence Pairs Dataset.
- [6] Papineni K, Roukos S, Ward T, Zhu W J. BLEU: a Method for Automatic Evaluation of Machine Translation. ACL, 2002.